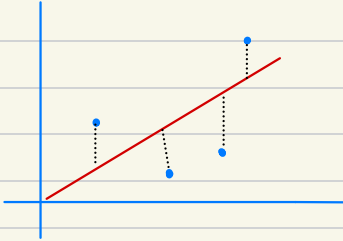




Gradient Descent

Also used in TSne, Linear Regression, Logistic Regression.



OLS : $L = \sum_{i=1}^n \sqrt{(y_i - \hat{y}_i)^2}$ should be less

$$\hat{y}_i = mx_i + b$$

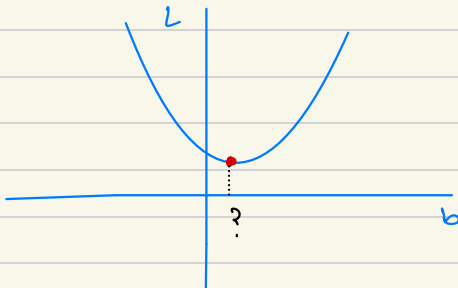
$$L = \sum_{i=1}^n \sqrt{(y_i - mx_i - b)^2}$$

$$L(m, b)$$

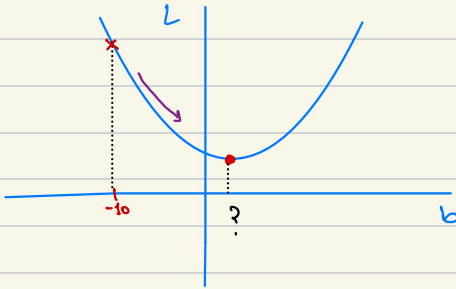
Let $m = 78.35$.

$$L = \sum_{i=1}^n \sqrt{(y_i - 78.35x_i - b)^2}$$

Value of b for which L is min.

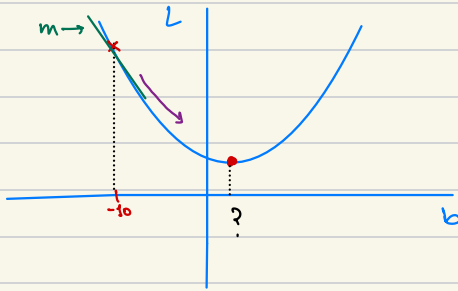


Step 1: Select a random b (let $b = -10$)



From $b = -10$ to point $?$

Algo. has 2 directions to move b .



Find slope (m) at that point & move to opposite direction.

$$b_{\text{new}} = b_{\text{old}} - \text{slope}$$

↓ Transform.

$$b_{\text{new}} = b_{\text{old}} - \eta \text{ Slope}$$

$\eta = 0.01$
(learning rate)

Helps in preventing
heavy shifts.

When to stop?

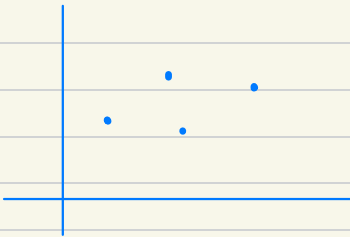
→ If $b_{\text{new}} - b_{\text{old}} = 0.0001$ or very low

In next iteration, there is no benefit in m .

There is no improvement, hence we converged.

→ Limiting the iteration. (epoch)

Mathematical Formulations.



Let $m = 70.35$

$b = ?$

Step 1: Start with random of b .

(Let $b = b$)

for i in epoch:

$b_{\text{new}} = b_{\text{old}} - \eta \text{ slope}$

(Let $\eta = 0.01$)

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Finding slope at $b = b$.

$$\frac{dL}{db} = \frac{d}{db} \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right)$$

$$\frac{d}{db} \left(\sum_{i=1}^n (y_i - mx_i - b)^2 \right) = 2 \sum_{i=1}^n (y_i - mx_i - b) (-1)$$

$$= -2 \sum_{i=1}^n (y_i - mx_i - b)$$

for $b > 0$

$$= -2 \sum_{i=1}^n (y_i - m x_i)$$

Let $m = 78.35$

$$= -2 \sum_{i=1}^n (y_i - 78.35 x_i)$$

$$b_{new} = b_{old} - \underbrace{\eta \times \text{slope}}_{\text{step-size}}$$

for i in range:

$$\hookrightarrow b_{new} = b_{old} - \eta \cdot \text{slope}$$

$$\hookrightarrow -2 \sum_{i=1}^n (y_i - 78.35 x_i - b)$$

Learning rate is very sensitive in Gradient Descent.

Adding m into the mix.

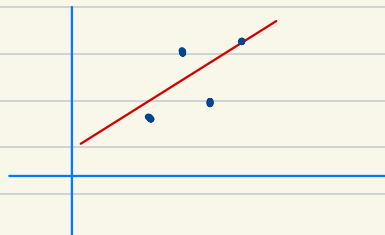
Step 1: Initialize random values for m and b .

Step 2: Define epochs = 100, learning-rate = 0.01.

for i in epochs:

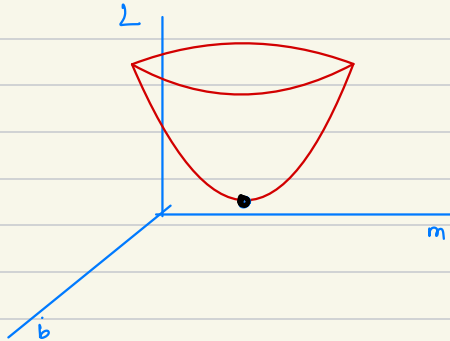
$$b = b - \eta \cdot \text{slope}_b$$

$$m = m - \eta \cdot \text{slope}_m$$



$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$L(m, b) = \sum_{i=1}^n (y_i - mx_i - b)^2$$



$$\frac{\partial L}{\partial b} = \frac{\partial}{\partial b} \sum_{i=1}^n (y - mx_i - b)^2$$

$$\text{slope-}b = \frac{\partial L}{\partial b} = -2 \sum_{i=1}^n (y - mx_i - b)$$

$$\frac{\partial L}{\partial m} = \frac{\partial}{\partial m} \sum_{i=1}^n (y - mx_i - b)^2$$

$$\text{slope-}m = \frac{\partial L}{\partial m} = -2 \sum_{i=1}^n (y - mx_i - b)(x_i)$$

Effect of Learning Rate :

Small $\eta \rightarrow$ more time to converge, takes more epochs.

Optimum $\eta \rightarrow$ less time to converge.

Large $\eta \rightarrow$ Zig-zag motion, risky

V. large $\eta \rightarrow$ high zig-zag motion, will never converge.

Effects of Loss Function :

$$L = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Conver fcn. $\rightarrow$ $\therefore$ Only 1 global minima.

Loss fcn may have saddle point $\rightarrow$ Long time for training.

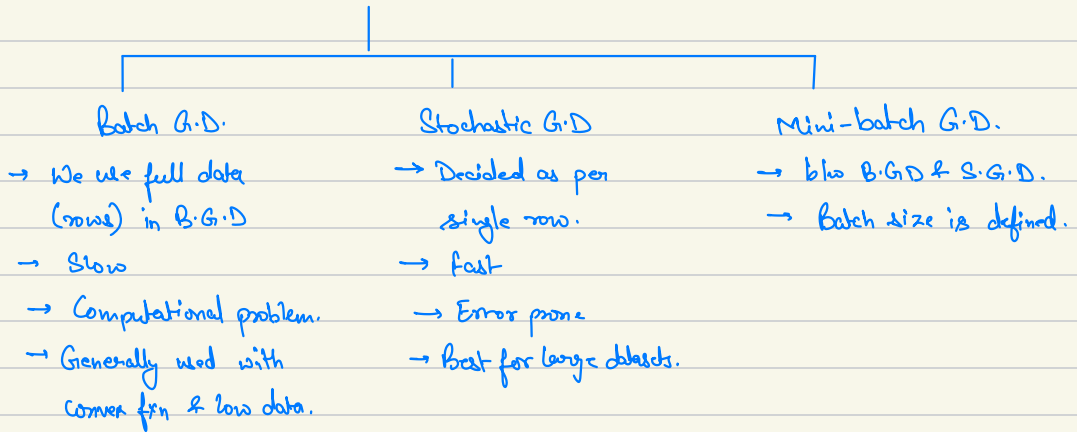
G.D is very sensitive to loss fcn.

Effect of data :

- If the features (col.) of data are at similar scale, we converge fast.
- But if feature scale is very different one in tens other in lakhs, we will converge very slow.
- We should scale our data before gradient descent.

Batch gradient descent

Gradient Descent.



cgpa | iq | gender | lpa.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3.$$

n-dims. $\downarrow$

$$\sum \beta_0, \beta_1, \dots, \beta_n$$

Mathematical Formulation of BGD :

cgpa	iq	lpa
x_1	x_2	y
8.1	93	3.2
7.5	95	3.5

(2x3)

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

$\nearrow$ β_1 $\nearrow$ β_2
 $\nearrow$ β_0 $\nearrow$ β_2

Finding $\rightarrow \{\beta_0, \beta_1, \beta_2\}$

Step 1 : Random values

$$\beta_0 (\text{intercept}) = 0 \quad (\text{Generally we use this initialization})$$

$$\beta_1, \beta_2 = 1$$

Step 2 : epoch = 100, $\eta = 0.1$

Update $\beta_0, \beta_1, \beta_2$

$$\beta_0 = \beta_0 - \eta \text{slope}_1 \quad \nearrow \frac{\partial L}{\partial \beta_0}$$

$$\beta_1 = \beta_1 - \eta \text{slope}_2$$

$$\beta_2 = \beta_2 - \eta \text{slope}_3$$

$$L(\beta_0, \beta_1, \beta_2)$$

$$\frac{\partial L}{\partial \beta_0}, \quad \frac{\partial L}{\partial \beta_1}, \quad \frac{\partial L}{\partial \beta_2}$$

for n dims $\rightarrow$ n+1 derivatives.

Calculating derivative of $\frac{dL}{d\beta}$

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{rows} = 2, \text{ columns} = 2+1$$

$\uparrow \quad \uparrow$
 $\text{ip} \quad \text{op}$

$$L = \frac{1}{2} \left[(y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 \right]$$

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

$$\hat{y}_1 = \beta_0 + \beta_1 x_{11} + \beta_2 x_{12}$$

$$\hat{y}_2 = \beta_0 + \beta_1 x_{21} + \beta_2 x_{22}$$

cgpa	x_1	x_2	gpa
y_{11}	8.1	x_{12}	3.2
y_{21}	7.5	x_{22}	3.5

(2x3)

$$L = \frac{1}{2} \left[(y_1 - \beta_0 - \beta_1 x_{11} - \beta_2 x_{12})^2 + (y_2 - \beta_0 - \beta_1 x_{21} - \beta_2 x_{22})^2 \right]$$

$$\frac{\partial L}{\partial \beta_0} = \frac{1}{2} \left[2(y_1 - \hat{y}_1)(-1) + 2(y_2 - \hat{y}_2)(-1) \right]$$

$$\frac{\partial L}{\partial \beta_0} = -\frac{2}{2} \left[(y_1 - \hat{y}_1) + (y_2 - \hat{y}_2) \right]$$

$$\frac{\partial L}{\partial \beta_0} = -\frac{2}{n} \left[(y_1 - \hat{y}_1) + (y_1 - \hat{y}_2) + \dots + (y_1 - \hat{y}_n) \right]$$

$$\frac{\partial L}{\partial \beta_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \rightarrow \text{slap 1 eqn.}$$

$$L = \frac{1}{2} \left[(y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 \right]$$

$$L = \frac{1}{2} \left[(y_1 - \beta_0 - \beta_1 x_{11} - \beta_2 x_{12})^2 + (y_2 - \beta_0 - \beta_1 x_{21} - \beta_2 x_{22})^2 \right]$$

$$\frac{\partial L}{\partial \beta_1} = \frac{1}{2} \left[2(y_1 - \hat{y}_1)(-x_{11}) + 2(y_2 - \hat{y}_2)(-x_{21}) \right]$$

$$\frac{\partial L}{\partial \beta_1} = \frac{-2}{n} \left[(y_1 - \hat{y}_1)x_{11} + (y_2 - \hat{y}_2)x_{21} + (y_3 - \hat{y}_3)x_{31} \dots \dots + (y_n - \hat{y}_n)x_{n1} \right]$$

$$\frac{\partial L}{\partial \beta_1} = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)(x_{i1}) \rightarrow \text{slope 2 eqn.}$$

$$\frac{\partial L}{\partial \beta_2} = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)(x_{i2})$$

$$\frac{\partial L}{\partial \beta_m} = \frac{-2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)(x_{im})$$

(General formula).

Problem with BGD :

1) $n = 10^5$

$\text{col} = 10^2$

$\text{epoch} \approx 10^3$

$\therefore \text{total computation} \rightarrow 10^{10}$

Very Slow.

2) Hardware $\rightarrow$ RAM

Stochastic Gradient descent

SGD $\rightarrow$ Row $\rightarrow$ update.

for n rows $\rightarrow$ 1 epoch $\rightarrow$ n updates.

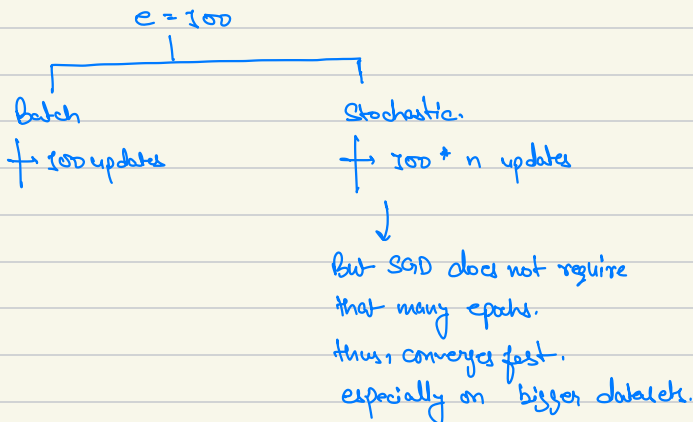
Due to frequent updates, we converge fast.

Stochastic : In SGD we pick rows randomly out of n rows.

Problem : It doesn't give us steady result.
Result vary every time after SGD.

Time comparison :

no. of epochs $\rightarrow$ fixed



When do we use SGD?

$\rightarrow$ Big Data

$\rightarrow$ Non-conver for.

Problem : Fluctuation of method (SGD) even at convergence point.

Learning schedule :

We vary our learning rate for every epoch.

$t_0, t_1 = 5, 50$

```
def learning_rate(t):  
    return  $t_0 / (t + t_1)$ 
```

```
for i in range(epochs):  
    for j in range(x.shape[0]):
```

```
        lr = learning_rate( $i * x.shape[0] + j$ )
```

Mini-Batch Gradient Descent

batch $\longrightarrow$ group of sizes
 $n = 1000$
 $\hookrightarrow$ batch = 100.
 $\therefore$ 10 batches.

batch
update | epoch